



Certified Kubernetes Administrator (CKA): Managing Cluster Lifecycle and Upgrades

Installing cluster components with Helm, Kustomize, CRDs, and operators



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com



Four framing questions



How do I install and version third-party software with Helm?



How do I tailor my own manifests per environment with Kustomize?



What are CRDs, and how do they extend the Kubernetes API?



How does an operator manage custom resources for me?





"The platform team needs metrics-server, an Ingress controller, and cert-manager this week -- none of it built in. And whatever we install, I want managed like a core resource: versioned, upgradable, rollback-able."

Hector S., Platform Engineering Lead, Globomantics Robotics Company



How do I install and version third-party software with Helm?



Helm: charts, releases, and the lifecycle



A chart is Helm's package: a bundle of templated Kubernetes manifests



A repository is a catalog of charts you add with `helm repo add`



Helm is the package manager for Kubernetes -- think apt or npm for clusters



Helm: charts, releases, and the lifecycle



helm install creates a release; helm upgrade revises it; helm rollback reverts it



helm history lists every revision; helm status shows the current one



A release is one installed instance of a chart, with its own name and history



The Helm lifecycle, end to end

```
# Add the OFFICIAL metrics-server repo (verified 2026-06-15), then refresh
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm repo update
```

```
# Install -> creates the release at revision 1
helm install metrics-server metrics-server/metrics-server -n kube-system \
  --version 3.12.2 --set args={--kubelet-insecure-tls}
```

```
# Upgrade -> revision 2 (newer chart); inspect, then undo
helm upgrade metrics-server metrics-server/metrics-server -n kube-system \
  --version 3.13.0 --reuse-values
```

```
helm history metrics-server -n kube-system # see revisions 1 and 2
helm rollback metrics-server 1 -n kube-system # revert -> new revision 3
```



How do I tailor my own manifests per environment with Kustomize?



Kustomize: bases, overlays, and apply -k

A base is your canonical manifests plus a kustomization.yaml listing them

An overlay points at the base and layers per-environment changes on top

Kustomize customizes plain YAML -- no templating language, it ships in kubectl

`kubectl apply -k <dir>` renders the kustomization, then applies the result

`kubectl kustomize <dir>` previews the rendered YAML without applying it

Overlays patch fields like replicas and set a namespace, leaving the base untouched



A staging overlay over a shared base

```
# overlays/staging/kustomization.yaml -- references the base, patches it
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: staging          # all resources land in the staging namespace
resources:
  - ../../base              # pull in the canonical Deployment + Service
patches:
  - target: {kind: Deployment, name: globomantics-app}
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 1              # staging runs lean: 1 replica, base stays at 2

# Preview, then apply -- no separate kustomize CLI needed
kubectl kustomize overlays/staging/  # render only
kubectl apply -k overlays/staging/   # render + apply
```



What are CRDs and operators, and how do they extend the API?



CRDs, custom resources, and operators

A CRD (Custom Resource Definition) registers a brand-new Kind with the API server

Once registered, a custom resource is just an object of that new Kind, like a Certificate

kubectl get crd lists them; kubectl api-resources shows every type, built-in and custom

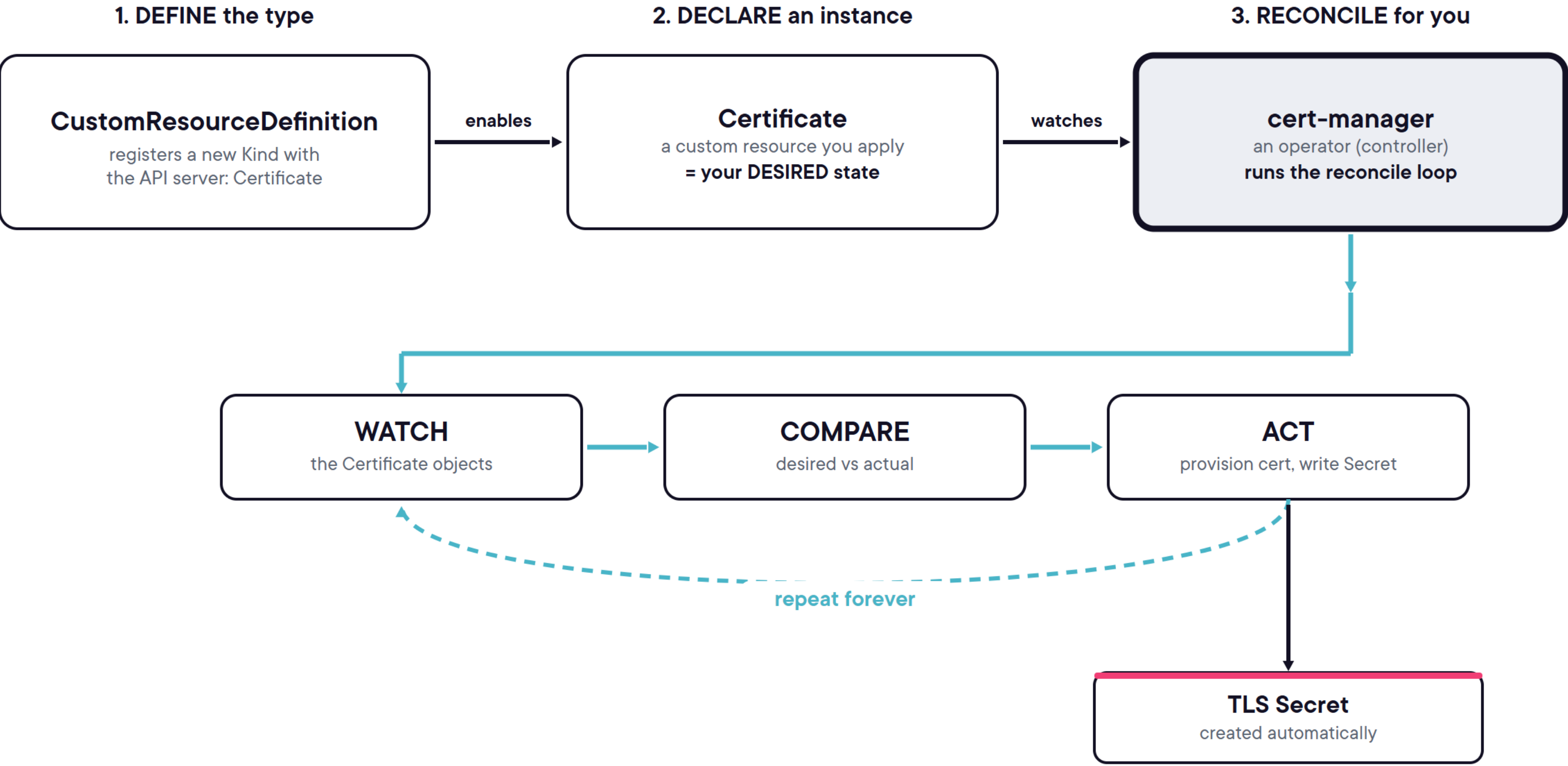
kubectl explain reads a CRD's schema to document its spec fields

An operator is a controller that watches custom resources and acts on them

Its reconciliation loop watches, compares desired vs actual, then acts to close the gap



CRD plus operator: the reconciliation loop



Install an operator, then inspect its custom resources

```
# Install cert-manager (VERIFIED v1.20.2) -- its manifest registers the CRDs
kubectl apply -f \
  https://github.com/cert-manager/cert-manager/releases/download/v1.20.2/cert-
manager.yaml
```

```
# The new Kinds the operator just added
kubectl get crd | grep cert-manager
kubectl api-resources | grep cert-manager    # certificates, issuers,
clusterissuers
```

```
# Read the schema with no internet, then watch the operator work
kubectl explain certificate.spec
kubectl get pods -n cert-manager            # the controller + webhook +
cainjector
kubectl get certificate                      # the custom resource the loop
manages
```



Demo

Helm, Kustomize, and an operator on the live cluster

On the Vagrant VM cluster: install and roll back a Helm chart, apply a Kustomize overlay, and stand up cert-manager to watch it manage a custom resource.





"The metrics-server, the Ingress controller, and cert-manager are all live -- each one versioned, upgradable, and rollback-able. When an upgrade misbehaves, it's one command back to safety."

Hector S., Platform Engineering Lead, Globomantics Robotics Company





From Globomantics to you



Helm: chart, repo, release -- install, upgrade, rollback, with history and status



Kustomize: one base, layered overlays, applied with
kubectl apply -k



CRDs add new Kinds; kubectl explain and api-resources prove they registered



Operators run a reconciliation loop -- watch, compare, act --
to manage your CRs





Course summary

Helm installs and versions third-party software as releases you can roll back

Kustomize tailors your own manifests per environment with `kubectl apply -k`

CRDs register new Kinds; `kubectl explain` and `api-resources` confirm them

Operators reconcile custom resources: watch, compare, act, repeat

Course 3 arc: back the cluster up, upgrade it safely, manage what runs on it





Thank you for your time!

Tim Warner | TechTrainerTim.com

